



1. (15 نمره)

الف - (5 نمره) شبه کدی بنویسید که دو رشته حرفی را با هم مقایسه کند، در صورتی که آن دو رشته برابر بودند مقدار 1 و در غیر این صورت 0 را چاپ کند (راهنمایی: در نوشتن شبه کد فرض کنید تابع len() را برای به دست آوردن طول یک رشته در اختیار دارید).

پاسخ:

```
input str1 and str2
initialize result to 1
if len(str1) != len(str2) then
    result = 0
else
    for i form 0 to len(str1) - 1 do
        if str1[i] != str2[i] then
            result = 0
        endif
    endfor
endif
print(result)
```

ب - (10 نمره) دو تابع زیر را کامل کنید. تابع getLength باید طول یک رشته را محاسبه و برگرداند و سپس با استفاده از این تابع، تابع compareStrings را طوری کامل کنید که دو رشته را مقایسه کند؛ در صورتی که این دو رشته برابر بودند مقدار 1 و در غیر این صورت مقدار 0 را برگرداند.

```
int getLength(char* str) {
    ...
}
```

```
int compareStrings(char* str1, char* str2){
    ...
}
```

پاسخ:

```
int getLength(char* str) {
    int len = 0, i = 0;

    while(str[i] != '\0'){
        ++len;
        ++i;
    }
    return len;
}

int compareStrings(char* str1, char* str2) {
    int len1 = getLength(str1);
    int len2 = getLength(str2);

    if(len1 != len2){
        return 0;
    } else {
        for(int i = 0; i < len1; i++){
            if(str1[i] != str2[i]){
                return 0;
            }
        }
    }
    return 1;
}
```

2. (20 نمره) فری کثیف صاحب یک ساندویچی در مرکز شهر است ولی متاسفانه مشتری زیادی ندارد، چون سطح بهداشت ساندویچ های فری کثیف بسیار بالاتر از استاندارد های بین المللی است. او برای راهکار جذب مشتری تصمیم میگیرد برای هر ساندویچ خود، یک تخفیف بگذارد تا تعداد مشتری ها افزایش یابد.

الف- (5 نمره) یک struct به نام Sandwich با فیلدهای زیر تعریف کنید:

- نام ساندویچ (رشته کاراکتری با طول نامشخص)
- قیمت ساندویچ (عدد صحیح)
- درصد تخفیف (عدد صحیح)

پاسخ:

```
typedef struct Sandwich {  
    char* name;  
    int price;  
    int discount;  
} Sandwich;
```

ب- (5 نمره) تابع searchSandwich را با استفاده از توابع نوشته شده بالا طوری کامل کنید که نام یک ساندویچ به همراه آرایه منوی ساندویچی را بگیرد و به دنبال غذای مورد نظر در منو بگردد. در صورتی که این غذا وجود داشت، آدرس آن خانه از آرایه را برگرداند و در غیر این صورت، NULL برگرداند.

*** توجه داشته باشید که برای این سوال استفاده از هیچ کتابخانه ای مجاز نیست.

در صورت نیاز به مقایسه رشته ها باید از توابعی که در سوال اول نوشتید استفاده کنید.

```
Sandwich* searchSandwich(char* name, Sandwich* dishes, int menu_size) {  
    ...  
}
```

پاسخ:

```
Sandwich* searchSandwich(char* name, Sandwich* dishes, int menu_size) {  
  
    for(int i = 0; i < menu_size; i++){  
        if(compareStrings(name, dishes[i].name)){  
            return dishes + i;  
        }  
    }  
  
    return NULL;  
}
```

پ- (10 نمره) اصغر که متوجه تخفیف های ساندویچی فری کشیف شده، تصمیم میگیرد تا ناهار را در این ساندویچی بخورد اما نمیداند که پول آن برای خرید یک ساندویچ مشخص کافی است یا نه. با کامل کردن تابع `canAfford` به اصغر کمک کنید تا بفهمد آیا با مقدار پولی که دارد، میتواند ساندویچ دلخواهش را بخرد یا نه. اول باید با استفاده از تابع قسمت الف بررسی کنید که آیا در منو، ساندویچ مدنظر اصغر وجود دارد یا نه. اگر وجود داشت بررسی کنید اگر پول اصغر کافی باشد، باید نام، قیمت اصلی و قیمت نهایی (بعد از اعمال تخفیف) که اصغر باید پرداخت کند چاپ شود. اگر پول کافی نبود یا ساندویچ وجود نداشت نیز پیام های مناسب طبق مثال ها چاپ شود.

```
void canAfford(int money, char* name, Sandwich* dishes, int menu_size){
}
```

مثال)

- اگر ساندویچ در منو وجود نداشت:

Sandwich not found

- اگر پول برای خرید ساندویچ کافی نباشد:

Money is not enough

- اگر پول به اندازه ساندویچ خواسته شده باشد: (اطلاعات غذا مطابق نمونه چاپ شوند)

Sandwich: bandari

Price: 170

Discounted Price: 144.500000

پاسخ:

```
void canAfford(int money, char* name, Sandwich* dishes, int menu_size) {
    Sandwich* selected = searchSandwich(name, dishes, menu_size);
    if(selected == NULL){
        printf("Sandwich not found\n");
        return;
    }

    float final_price = (float)selected->price*(100-selected->discount)/100;
    if(final_price > money){
        printf("Money is not enough\n");
    } else {
        printf("Sandwich: %s\nPrice: %d\nDiscounted Price:%f",
            selected->name, selected->price, final_price);
    }
}
```

3. (15 نمره) می‌خواهیم خانه‌های یک شهر را پلاک‌گذاری کنیم طوری که هیچ دو خانه‌ای پلاک یکسان نداشته باشند.

برای این کار، یک "دفتر ثبت پلاک‌ها" داریم که حداکثر می‌تواند اطلاعات MAX خانه را نگه دارد.

```
#define MAX 100
```

...

الف – (2.5 نمره) یک struct به نام House تعریف کنید که بیانگر یک خانه با فیلدهای زیر است:

- شماره پلاک (عدد صحیح)
- نام خیابان (رشته کاراکتری با حداکثر طول 50 کاراکتر)
- شماره واحد (عدد صحیح)

پاسخ:

```
#define STREET_NAME_MAX_LEN 50

typedef struct House{
    int plate;
    char street[STREET_NAME_MAX_LEN];
    int unit;
} House;
```

ب – (2.5 نمره) یک struct دیگر به نام Registry تعریف کنید شامل:

- آرایه‌ای از House با اندازه MAX
- تعداد خانه‌های ثبت‌شده تا این لحظه

پاسخ:

```
#define MAX 100

typedef struct Registry{
    House houses[MAX];
    int count;
} Registry;
```

پ- (10 نمره) تابع زیر را طوری کامل کنید که اشاره گر به یک رجیستری و اطلاعات یک خانه جدید را بگیرد. اگر پلاک قبلاً ثبت نشده بود، خانه را به انتهای آرایه رجیستری اضافه کند و مقدار 1 برگرداند. اگر پلاک تکراری بود، هیچ چیزی اضافه نکند و مقدار 0 برگرداند؛ و اگر رجیستری پر بود، مقدار 1- را برگرداند.

راهنمایی: تابع strcpy یک تابع در کتابخانه <string.h> است. این تابع دو رشته را به عنوان آرگومان میگیرد و رشته دوم را در رشته اول کپی می کند. در صورت نیاز می توانید از این تابع استفاده کنید.

```
int registerHouse(Registry* reg, int plate, char street[], int unit){
    ...
}
```

پاسخ:

```
#include <string.h>

int registerHouse(Registry* reg, int plate, char street[], int unit){

    if (reg->count >= MAX) {
        return -1;
    }

    for (int i = 0; i < reg->count; i++) {
        if (reg->houses[i].plate == plate) {
            return 0;
        }
    }

    reg->houses[reg->count].plate = plate;
    strcpy(reg->houses[reg->count].street, street);
    reg->houses[reg->count].unit = unit;

    reg->count++;
    return 1;
}
```

4. **صرفا برای یادگیری (این سوال جزو سوالات تمرین هشتم نبوده و صرفا یک مثال

برای یادگیری شماست)

غضنفر که در دانشگاه تهران تحصیل میکند، متوجه میشود که چیزی تا پایان ترم نمانده! او که کل ترم را در سایت قدیم دانشکده سپری کرده و وقت ارزشمندش را تلف کرده، تصمیم میگیرد به طور جدی درس را شروع کند، برای همین به سراغ کتاب های درسی خود، واقع در کتابخانه اتاقش میرود. کتابخانه او را آرایه ای از اشاره گر ها به کتاب (Book) در نظر بگیرید. (توجه داشته باشید که همه عناصر این آرایه لزوما یک اشاره گر به Book نیستند و ممکن است عنصری به NULL اشاره کند؛ یعنی جای آن کتاب در کتابخانه خالی باشد. همچنین حافظه های مربوط به Book، پویا نیستند و نیازی به آزاد سازی حافظه ندارند)

الف- یک struct به نام Date برای تاریخ تعریف کنید که شامل فیلدهای روز، ماه، سال (هر سه عدد صحیح) باشد.

پاسخ:

```
typedef struct Date {
    int day;
    int month;
    int year;
} Date;
```

ب- یک struct دیگر به نام Book برای کتاب تعریف کنید که شامل فیلدهای زیر باشد:

- نام کتاب (یک رشته کاراکتری به صورت char*)
- موضوع کتاب (یک رشته کاراکتری به صورت char*)
- تعداد صفحات (عدد صحیح)
- تاریخ چاپ کتاب (از نوع Date)

پاسخ:

```
typedef struct Book {
    char* name;
    char* subject;
    int pages;
    Date publication_date;
} Book;
```

پ-غضنفر به یاد می‌آورد که استاد گفته بود کتاب های قدیمی به درد نمیخورد! تابع `checkDate` را طوری کامل کنید که 2 تاریخ را بگیرد و اگر تاریخ مورد نظر از تاریخ مرجع، جدیدتر بود، مقدار 1 و در غیر این صورت مقدار 0 را برگرداند.

```
int compareDates(Date reference_date, Date target_date)
...
}
```

پاسخ:

```
int compareDates(Date reference_date, Date target_date) {

    if(reference_date.year < target_date.year){
        return 1;
    }
    else if(reference_date.year > target_date.year){
        return 0;
    }
    else { // years are equal
        if(reference_date.month < target_date.month){
            return 1;
        }
        else if(reference_date.month > target_date.month){
            return 0;
        }
        else { // months are also equal
            if(reference_date.day < target_date.day){
                return 1;
            }
            else{
                return 0;
            }
        }
    }
}
```

ت-تابع زیر را طوری کامل کنید که به غضنفر کمک کنید تا کتاب هایی که جدیدتر از تاریخ مرجع نیستند را حذف کند(از تابع قسمت قبل کمک بگیرید). این تابع یک آرایه ای از اشاره گر ها به `Book` میگیرد و در صورت حذف یک کتاب، عنصر مرتبط با آن کتاب باید به `NULL` اشاره کند.

```
void deleteOldBooks(Book** library, Date reference_date, int library_size) {
}
```


پاسخ:

```
void deleteOldBooks(Book** library, Date reference_date, int library_size) {  
    for(int i = 0; i < library_size; i++){  
        if(library[i] != NULL){ // this comparison is too important!  
            if(!compareDates(reference_date, library[i]->publication_date)){  
                library[i] = NULL;  
            }  
        }  
    }  
}
```

ث-غضنفر که مدت زمان زیادی برایش باقی نمانده، میخواهد بداند که کدام کتاب، بیشترین زمان را جهت مطالعه کردن میگیرد.

زمان مورد نیاز جهت مطالعه هر کتاب، از این رابطه به دست می آید:

ضریب درس * تعداد صفحات درس = زمان مورد نیاز

غضنفر فقط 3 درس ریاضی ("Math") با ضریب 4، شیمی ("Chemistry") با ضریب 3 و ادبیات ("Literature") با ضریب 2 را دارد.

تابع زیر را طوری کامل کنید که زمان مورد نیاز برای مطالعه یک کتاب را محاسبه کند. در صورت نیاز به مقایسه رشته ها، از توابعی که در سوال یک نوشتید استفاده کنید.

```
int calculateTimeCost(Book* selected) {  
    ...  
}
```

پاسخ:

```
#define MATH_UNIT 4  
#define CHEMISTRY_UNIT 3  
#define LITERATURE_UNIT 2  
  
int calculateTimeCost(Book* selected) {  
    int credit;  
    // Using function "compareStrings" from question 1  
    if(compareStrings(selected->subject, "Math")){  
        credit = MATH_UNIT;  
    }  
}
```

```

else if(compareStrings(selected->subject, "Chemistry")){
    credit = CHEMISTRY_UNIT;
}
else {
    credit = LITERATURE_UNIT;
}

return credit * selected->pages;
}

```

ج- تابع زیر را طوری کامل کنید که با توجه به فرجه امتحانی غضنفر برحسب روز (respite)، آدرس کتابی که بیشترین زمان را مصرف میکند برگرداند. (کتابی که مدت زمان مطالعه بیشتر از فرجه اختیار میکند قابل قبول نیست. در صورت وجود نداشتن هیچ کتابی، مقدار NULL برگرداند).

```

Book* findTheHardestChoice(Book** library, int respite, int library_size) {
    ...
}

```

پاسخ:

```

Book* findTheHardestChoice(Book** library, int respite, int library_size) {
    Book* worst_choice = NULL;
    int worst_time_cost;

    for(int i = 0; i < library_size; i++){
        if(library[i] != NULL){ // this comparison is too important!
            int time_cost = calculateTimeCost(library[i]);

            if(time_cost <= respite){
                if((worst_choice == NULL) || (time_cost > worst_time_cost)){
                    worst_choice = library[i];
                    worst_time_cost = time_cost;
                }
            }
        }
    }

    return worst_choice;
}

```

4. **امتیازی (20 نمره) صف (Queue) در برنامه نویسی یک ساختمان داده خطی است که

داده‌ها را به ترتیب ورود و خروجشان مدیریت می‌کند. این ترتیب بر اساس اصل **اولین ورودی، اولین خروجی (FIFO)** تعیین می‌شود؛ مانند صف نانویی که اولین فردی که وارد صف می‌شود، اولین فردی است که نوبتش می‌رسد و خارج می‌گردد (به جز احتمالاً در ایران 😊). این ساختمان داده برای مدیریت وظایف، پردازش درخواست‌ها و ارتباطات سیستم‌ها کاربرد فراوان دارد.

یک سیستم پشتیبانی، پیام‌های کاربران را به ترتیب ورود نگه می‌دارد تا اپراتور به ترتیب به آن‌ها رسیدگی کند. شما باید یک صف ساده بسازید که عملیات اصلی صف را پشتیبانی کند.

ویژگی‌های صف:

- هر پیام شامل: شناسه پیام، نام فرستنده و متن پیام است.

- صف ظرفیت ثابت CAP دارد.

#define CAP 10

- صف با **آرایه‌ی خطی ساده** پیاده‌سازی می‌شود. بنابراین هنگام حذف یک عنصر از ابتدای صف (عملیات حذف همواره از ابتدای صف اتفاق می‌افتد)، باید عناصر باقی‌مانده یک خانه به سمت چپ جابه‌جا شوند تا ترتیب حفظ شود.

الف- (2.5 نمره) یک struct به نام Message با فیلدهای زیر تعریف کنید که بیانگر یک پیام است.

- شناسه (id) به صورت عدد صحیح

- نام فرستنده به صورت رشته (حداکثر 20 کاراکتر)

- متن پیام به صورت رشته (حداکثر 100 کاراکتر)

پاسخ:

```
#define SENDER_NAME_MAX_LEN 20
#define TEXT_MAX_LEN 100

typedef struct Message {
    int id;
    char sender[SENDER_NAME_MAX_LEN];
    char text[TEXT_MAX_LEN];
} Message;
```

ب- (2.5 نمره) یک struct به نام Queue با فیلدهای زیر تعریف کنید که بیانگر صف پیام ها است:

- آرایه پیامها

- اندیس محل درج بعدی

- تعداد عناصر فعلی صف

پاسخ:

```
#define CAP 10

typedef struct Queue {
    Message msgs[CAP];
    int next;    // index of next insertion
    int count;   // current number of elements in queue
} Queue;
```

پ- (2.5 نمره) تابع زیر را طوری کامل کنید که یک صف تازه ساخته شده را آماده استفاده کند.

این تابع باید اندیس و شمارنده را به طور مناسب مقداردهی کند.

```
void initQueue(Queue* q) {
    ...
}
```

پاسخ:

```
void initQueue(Queue* q) {
    q->next = 0;
    q->count = 0;
}
```

ث- (2.5 نمره) دو تابع زیر را طوری کامل کنید که خالی بودن یا پر بودن صف را بررسی کنند.

```
int isEmpty(Queue* q) {
}

int isFull(Queue* q) {
}
```

پاسخ:

```
int isEmpty(Queue* q) {
    return (q->count == 0);
}
int isFull(Queue* q) {
    return (q->count == CAP);
}
```

ج- (5 نمره) تابع زیر را طوری کامل کنید که یک پیام جدید را در جای مناسب در صف اضافه کند. در صورت درج موفق، تابع باید مقدار 1 را برگرداند؛ اگر صف پر بود، تابع باید مقدار 0 را برگرداند.

```
int enqueue(Queue* q, Message msg) {
}
```

پاسخ:

```
int enqueue(Queue* q, Message msg) {
    if (isFull(q))
        return 0;
    q->msgs[q->next] = msg;
    q->next++;
    q->count++;
    return 1;
}
```

چ- (5 نمره) تابع زیر را طوری کامل کنید که پیام اول صف حذف کرده و آن را برگرداند. در صورتی که صف خالی باشد، تابع باید Null برگرداند.

```
Message* dequeue(Queue* q) {
}
```

پاسخ:

```
Message* dequeue(Queue* q) {
    if (isEmpty(q))
        return NULL;

    Message* removed = (Message*)malloc(sizeof(Message));
    *removed = q->msgs[0];
    for (int i = 1; i < q->count; i++) {
        q->msgs[i-1] = q->msgs[i];
    }
    q->next--;
    q->count--;
    return removed;
}
```